



BLUE BITS

البرمجة التفرعية

نظري (8)

ملف داعم

2026/2025

السنة الرابعة



مكتبة شهاب
للشعاع النيرة - يوق يلقى على القاصي

[f](#) [@](#) [v](#) [t](#) /BlueBitsTeam

www.bluebitsteam.com



لدينا اثنتين من التقنيات الأساسية في البرمجة التفرعية:

1. التقسيم *Partitioning*
2. فرق تسد *Divide – and – conquer*

وهذه التقنيات مرتبطة ببعضها.

- في التقسيم تنقسم المشكلة إلى أجزاء منفصلة ويحسب كل جزء بشكل منفصل.
- فرق تسد عادة يطبق التقسيم بشكل تعاودي أي تقسيم المشكلة باستمرار إلى أجزاء أصغر وأصغر قبل حلها وإعادة التجميع بين النتائج.

ملاحظة:

أمثلة عن التقسيم

Send

Broadcast

Reduce, Scatter

استراتيجيات التقسيم:

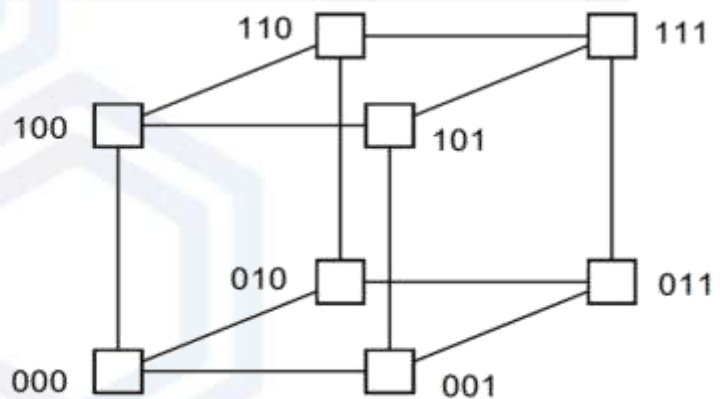
- التقسيم يقسم المشكلة إلى أجزاء
- المشاكل التفرعية الحرجة تستخدم التقسيم بدون أي تفاعل بين الأجزاء
- لكن معظم تركيبات التقسيم تتطلب نتائج الأجزاء المراد دمجها للحصول على النتيجة المطلوبة
- يمكن تطبيق التقسيم على بيانات البرنامج

أي تقسيم البيانات والعمل على البيانات المقسمة بشكل متزامن وهذا ما يسمى تقسيم البيانات أو تحليل المجال

يمكن تطبيق التقسيم على توابع البرنامج لتقسيمها إلى وظائف مستقلة وتنفيذها بشكل متزامن وهذا ما يعرف بالتحليل الوظيفي

Parallel Implementation (cont.)

We can see that these constructions are the same as the binary hypercube broadcast and gather algorithms.



كل رأسين متتاليين يختلفان بخانة فقط
نقوم بالتقسيم من قبل ال $processes$ ذات الخانة الأعلى
ونقوم بالتجميع من قبل ال $processes$ ذات الخانة الأدنى

شرح فكرة الخوارزميتين السابقتين:

الفكرة الرئيسية هي أن الاتصال يحدث بين العمليات المجاورة لبعضها $adjacent$ في المكعب المجاور، مما يسمح بتبادل البيانات الفعال.

خوارزمية $Binary Hypercube Broadcast Algorithm$:

1. تحمل عملية الجذر $root$ في البداية البيانات التي يجب توزيعها على جميع العمليات (العقد) الأخرى.
2. تبدأ عملية الجذر بإرسال البيانات إلى جيرانها المباشرين $neighbours$ في $Hypercube$.
3. كل عملية تستقبل البيانات تقوم بتوجيهها إلى جيرانها حتى تتلقى جميع العمليات البيانات.

الخوارزمية $Binary Hypercube Gather$:

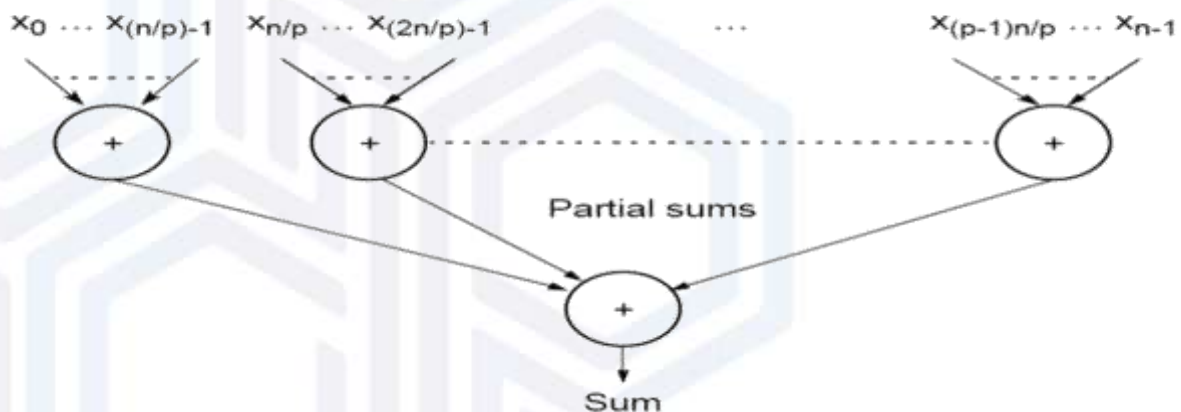
1. تحمل كل عملية جزءاً من البيانات التي يجب جمعها في عملية واحدة (عملية الجذر).
2. ترسل كل عملية بياناتها إلى جارتها المباشرة في $Hypercube$.
3. يتم إعادة توجيه البيانات من خلال $Hypercube$ حتى تصل إلى عملية الجذر، والتي تقوم بتجميع وجمع كل البيانات المستلمة.

مثال: لنفترض سيتم إضافة سلسلة من الأعداد $X_0, \dots, X_{n-1}$

الحل التفرعي لن يكون مفيد جداً

- ومع ذلك يمكن استخدام هذا النهج لتطبيقات أكثر واقعية تنطوي على حسابات معقدة او على قواعد بيانات كبيرة
- قد نفكر في تقسيم السلسلة إلى P جزء من الأعداد n/p
- يمكن لكل معالج (أو معالجة) جمع تسلسل واحد من الأرقام بشكل مستقل و إنشاء المجاميع الجزئية $partial sums$
- يجب جمع جميع المجاميع الجزئية الخاصة بكل المعالجات معاً لتكوين المجموع النهائي

The p partial sums need to be added together to form the final sum.





- كما هو الحال مع خوارزمية *broadcast/gather hypercube*
- المعالجات تستطيع أن تتواصل مع معالجات أخرى وتختلف عن بعضها في بت واحد فقط بدءاً من البت الأكثر أهمية لمرحلة التقسيم ومن البت الأقل أهمية لمرحلة الجمع
- لنفترض أننا أنشأنا ثماني معالجات بشكل ثابت لإضافة ملف قائمة الأرقام

Master – Slave approach (Example):

- الأرقام ترسل من المعالج *master* إلى المعالجات *slaves*
- تقوم ال *slaves* بجمع أرقامها، ويتم تنفيذها بشكل مستقل وتفرعي، وإرسال مجاميعها الجزئية إلى المعالج *master*
- يضيف (يجمع) المعالج الرئيسي *master* المجاميع الجزئية لتشكيل النتيجة النهائية
- لدينا نقطة خلافية فيما إذا كان سيتم عمل *Broadcast* القائمة بأكملها لكل *slave* أو فقط إرسال الأرقام المحددة لكل *slave* أيهما يكون الأفضل، لأنه في كلتا الحالتين يجب إرسال كافة الأرقام من *master*
- سيكون لل *Broadcast* وقت تشغيل واحد *single startup time* وليس عدّة أوقات تشغيل منفصلة عند استخدام إجراءات إرسال متعددة وقد نعتبر ذلك الأفضل

Partitioning (Example):

- أولاً سنقوم بإرسال الأرقام المحددة لكل *slave* باستخدام تابع *send()* الفردي
- إذا كانت الأعداد *n* مقسمة إلى مجموعات *p* حيث n/p عدد صحيح، ويتم تعيين المجموعة الواحدة لعملية تابعة *slave* واحدة، مما يؤدي إلى وجود عدد *p* من ال *slaves*

مثلاً: $n = 1000, \quad p = 10, \quad S = n/p$

عدد الأرقام في كل مجموعة: $S = 100$

مجموعة الأرقام لعملية *slave* الأولى $0 \rightarrow 100$

مجموعة الأرقام لعملية *slave* الثانية $101 \rightarrow 200$

Example (cont.) (Partitioning)

```

1 Master
2 s = n/p; /* number of numbers for slaves */
3 for (i = 0, x = 0; i < p; i++, x = x + s)
4 send (&numbers [x], s, Pi); /* send s numbers to slave */
5 sum = 0;
6 for (i = 0; i < p; i++)
7 { recv(&part sum, PANY); /* wait for results from slaves */
8   sum = sum + part sum; /* accumulate partial sums */

```



9 }

شرح:

يقوم الـ *Master* بتقسيم لائحة الأرقام إلى مجموعات وإرسالها بشكل فردي لكل *slave*، ثم يقوم باستقبال المجاميع الجزئية التي تم جمعها من قبل كل *slave* ويقوم بجمع هذه المجاميع الجزئية

ملاحظات:

تجزئة *Send, receive*

Master

سوف يقوم بعملية *Send* بحلقة تكرار عملية الإرسال للـ *Pi* وبعدها استقبال عدد من المرات

```

1 Slave
2 recv(numbers, s, Pmaster); /* receive s numbers from master */
3 part_sum = 0;
4 for (i = 0; i < s; i++) /* add numbers */
5 part sum = part_sum + numbers[i]; جمع العناصر
6 send(&part_sum, Pmaster); /* send sum to master */

```

: Broadcast

يتم استخدام *broadcast* لإرسال القائمة الكاملة لكل *slave* هناك حاجة إلى الكود في كل *slave* لتحديد الجزء من السلسلة المراد استخدامها من قبل الـ *slave*، مما يضيف خطوات حساب إضافية داخل العملية *slave*

Master

```

s = n/p; /* number of numbers for slaves */
bcast(numbers, s, Pslave_group); /* send all numbers to slaves */
sum = 0;
for (i = 0; i < p; i++) { /* wait for results from slaves */
    recv(&part_sum, PANY);
    sum = sum + part_sum; /* accumulate partial sums */
}

```

أرسلت المصفوفة
للجميع

سوف يستقبل
بعدد المعالجات

Broadcast سوف تكون موجودة عند كل من الـ *master* والـ *slave*



تستقبل المصفوفة
من *master*

الجزء الذي سأعمل
عليه

Slave

```
bcast(numbers, s, Pmaster);          /* receive all numbers from master */
start = slave_number * s;           /* slave number obtained earlier */
end = start + s;
part_sum = 0;
for (i = start; i < end; i++)        /* add numbers */
    part_sum = part_sum + numbers[i];
send(&part_sum, Pmaster);           /* send sum to master */
```

$$P_0: \text{start} = 0 * 100 = 0 \quad \text{end} = 0 + 100 = 100$$

$$P_1: \text{start} = 1 * 100 = 100 \quad \text{end} = 100 + 100 = 200$$

يتم التعرف على *slave* من خلال معرفة معرفها الخاص *ID* والذي يمكن الحصول عليه باستدعاء مكتبة مخصصة

- في أغلب الأحيان، سيتم أولاً تشكيل مجموعة من العمليات ويكون رقم الـ *slave* عبارة عن *instance* أو *rank* داخل المجموعة.

Rank أو *Instance*: عبارة عن عدد صحيح من 0 إلى $m - 1$ حيث توجد m من العمليات في المجموعة.

- يتطلب *MPI* إنشاء *Communicator* وأن تكون العمليات لها مرتبة داخل جهاز الاتصال.

- يمكن ربط المجموعات ضمن الـ *Communicators*. ويكون للعمليات مرتبة داخل هذه المجموعات.

If scatter and reduce routines are available, as in MPI, the code could be:

Master

```
s = n/p;                                /* number of numbers */
scatter(numbers, &s, Pgroup, root=master); /* send numbers to slaves */
reduce_add(&sum, &part_sum, Pgroup, root=master); /* results from slaves */
```

Slave

```
scatter(numbers, &s, Pgroup, root=master); /* receive s numbers */
for(i = 0; i < p; i++)                    /* add numbers */
    part_sum = part_sum + number[i];
reduce_add(&sum, &part_sum, Pgroup, root=master); /* send sum to master */
```

يتم التقطيع وترسل
كل جزء لمعالجته

تستقبل. وتضع
الناتج بالـ *sum*

حلقة على عدد
العناصر وجمعهم

بعد أن تم الجمع
ترسل للـ *master*



ملاحظة:

حللنا مثال نقوم فيه بجمع مجموعة من الأرقام، ويمكن إجراء العديد من العمليات الأخرى بدلاً من ذلك

على سبيل المثال:

1. يمكن العثور على العدد الأكبر في المجموعة $max\ num$ وإعادته إلى الـ $Master$ حتى يتمكن من العثور على العدد الأكبر ضمن جميع العمليات التي مرّت إليه.
2. يمكن العثور على عدد مرات ظهور رقم (أو حرف، أو سلسلة من الأحرف) في مجموعات وإعادته إلى الـ $Master$.

تحليل الخوارزمية:

- يتطلب الحساب التسلسلي $n - 1$ إضافة مع تعقيد زمني قدره $O(n)$.
- في التنفيذ الموازي، هناك عدد p من الـ $slaves$.
- لتحليل التنفيذ التفرعي، سنفترض أن عمليات المعالجة الرئيسية $Master$ متضمنة في أحد المعالجات التابعة $slaves$ في نموذج $SPMD$.

المرحلة الأولى: الاتصال

أولاً، يجب الأخذ بالحسبان جانب الاتصال لعدد p من العمليات التابعة التي تقرأ كل منها مجموعة الأعداد الخاصة بها n/p

- يطلب استخدام إجراءات الإرسال والاستقبال وقت اتصال قدره:

$$t_{comm1} = p (t_{startup} + (n/p)t_{data})$$

حيث: $t_{startup}$ هو الجزء الزمني الثابت للإرسال

t_{data} هو الوقت المناسب لإرسال كلمة بيانات واحدة

- استخدام الـ $scatter$ قد يؤدي إلى تقليل عدد مرات بدء التشغيل

$$t_{comm1} = t_{startup} + n * t_{data}$$

التعقيد الزمني لا يزال $O(n)$

المرحلة الثانية: الحساب

بعد ذلك نحتاج إلى تقدير عدد الخطوات الحسابية

تجمع كل من العمليات الـ $slaves$ المجموعات $n/p - numbers$ معاً مما يتطلب إضافة $\frac{n}{p} - 1$ في حين كل العمليات التابعة يتم تنفيذها معاً، هذا يعني أنّ المجاميع الجزئية يتم الحصول عليها في الخطوة

$$\frac{n}{p} - 1$$

أي مثلاً إذا عملت 99 عملية جمع ولكن:

$$n/p = 100 \rightarrow 100 - 1$$

$$t_{comp} = \frac{n}{p} - 1$$



المرحلة الثالثة: الاتصال

إرجاع النتائج الجزئية باستخدام إجراءات الإرسال والاستقبال لديها وقت اتصال قدره:

$$t_{comm2} = P(t_{startup} + P * t_{data})$$

عند استخدام *Gather & Reduce* نحصل على اتصال قدره:

$$t_{comm2} = t_{startup} + p * t_{data}$$

المرحلة الرابعة: الحساب

بالنسبة للتراكم النهائي يجب على الـ *master* إضافة المجاميع الجزئية التي عددها P الأمر الذي يتطلب خطوات $p - 1$:

$$t_{comp} = p - 1$$

يجمع من المعالجات ويقوم بالعملية الحسابية

التحليل:

وقت التنفيذ الإجمالي *Overall Execution Time*:

$$\begin{aligned} tp &= (t_{comm1} + t_{comm2}) + (t_{comp1} + t_{comp2}) \\ &= p \left(t_{startup} + \left(\frac{n}{p} \right) t_{data} \right) + P(t_{startup} + t_{data}) + \left(\frac{n}{p} - 1 + p - 1 \right) \\ &= 2pt_{startup} + (n + p)t_{data} + p + \frac{n}{p} - 2 \end{aligned}$$

$$\text{or } tp = O(n)$$

تتكرر 100 مرة

master

$$t_{comm1} = ((n/p) t_{data} + t_{startup}) * p$$

ناتج الجمع لعنصر واحد

slave

$$t_{comm2} = (t_{data} + t_{startup}) * p$$

ملاحظة:

لعدد محدد من المعالجات. نرى أن التعقيد الزمني التفرعي هو نفس التعقيد الزمني التسلسلي وهو $O(n)$

وبطبيعة الحال، إذا أخذنا في الاعتبار الجانب الحسابي فقط، فإن الصيغة التفرعية أفضل من الصيغة التسلسلية.



"في الوقت متسع , وفيك عزيمة ... والدرب ممتد , وأنت شباب"

2026/2025

السنة الرابعة



BLUE BITS



أعضاء الفريق



الفريق التقني



الفريق العلمي

إبراهيم هبرة	عائشة العُمر العلّوش	إيناس بيلوني	ظلال بابا
إسراء حاج حمدي	عمر قظان	تسنيم طالب	عائشة قلّه
بتول لبّان	فاطمة الحسين	حلا إبراهيم	عائشة مُصطفى
بشرى كوكش	قاسم يحيى	دلال تسقيه	قمر كمد
بنيامين حكمان	كريست ياغجيان	ديانا صبّاغ	محمد يمان جلال
تسنيم حوكان	محمد منير	زهير حّقامي	محمود غزال
تسنيم نور الدّين	مديحة جّمّال	شكرية سُلطان	وفاء حّقامي
جميل هلال	ميرا فضلي	شهد كرّوم	يوسف نيّال
جودي جلال	نجم الدّين درويش		
خديجة بيازيد	هبة الله كتلو		
رشا معراوي	وداد قلعيّة		
سدرة مبيض	ياسر صبّاغ		

هل لديك أي ملاحظة؟ لا تتردّد في مراسلتنا.

